

NOTES

Pandas Basics



✓ Pandas Basics

✓ Understanding Series and DataFrames

Pandas is a Python library that provides powerful tools for data analysis and manipulation. It has two main data structures:

- Series
- Dataframe

Think of them as containers to store data in an organized way. For beginners, it's helpful to think of a Series as a column of data, and a DataFrame as a whole table with rows and columns.

✓ 1.What is a Pandas Series?

A Series is a one-dimensional array, like a single column of data in a spreadsheet. Each element in a Series has an index (like a label) that makes it easy to find specific values.

Key Points about Series: **One-dimensional:** A Series only has one column of data, so it's simpler than a DataFrame.

Indexed: Each value has an index label, so you can access values by their label.

Holds one data type: All values in a Series are usually of the same data type (e.g., all integers, all floats, or all strings).

When to Use Series:

- When you need to store a single column or list of data.
- For simpler datasets where you only need one type of information, like a list of ages or prices.

✓ a. Creating a Series in Pandas:

To create a Series, we can use the **pd.Series()** function. Let's look at different ways to create a Series.

Example 1: Creating a Series from a List

```
import pandas as pd
```

```
# Creating a Series from a list of numbers
data = [10, 20, 30, 40]
```

```
series = pd.Series(data)
print("Series with Default Index:\n", series)
```

```
➡ Series with Default Index:
0    10
1    20
2    30
3    40
dtype: int64
```

Explanation:

We used a list [10, 20, 30, 40] to create the Series.

Pandas automatically assigns default indices (0, 1, 2, 3).

Example 2: Creating a Series with a Custom Index

We can also assign custom labels (or indices) to each element in the Series.

```
# Creating a Series with custom index labels
labels = ['a', 'b', 'c', 'd']
series = pd.Series(data, index=labels)
print("Series with Custom Index:\n", series)
```

```
➡ Series with Custom Index:
a    10
b    20
c    30
d    40
dtype: int64
```

Explanation:

The list ['a', 'b', 'c', 'd'] specifies custom labels for the Series.

✓ b. Accessing Values in a Series:

You can retrieve specific values by using their index labels.

```
# Accessing a value by its label
print("Value at index 'b':", series['b']) # Output: 20
```

```
➡ Value at index 'b': 20
```

✓ 2.What is a Pandas DataFrame?

A DataFrame is a two-dimensional data structure, like a table, where data is organized in rows and columns. Each column can have a different data type (e.g., integers, floats, strings).

Key Points about DataFrames:

- Two-dimensional: A DataFrame has rows and columns, so it's like a table.
- Labeled rows and columns: Rows and columns both have labels, making it easy to access data.
- Multiple data types: Each column can store different types of data (e.g., numbers, text).

When to Use DataFrames:

- When you need to store data in a table format, with multiple columns representing different attributes.
- For more complex datasets where each column represents a different variable or feature.

✓ a. Creating a DataFrame in Pandas

To create a DataFrame, you can use the `pd.DataFrame()` function with different types of data sources.

```
# Creating a DataFrame from a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
print("DataFrame Example:\n")
df
```

🔄 DataFrame Example:

	Name	Age	City
0	Alice	24	New York
1	Bob	27	Los Angeles
2	Charlie	22	Chicago

Explanation:

We used a dictionary where each key (e.g., Name, Age, City) represents a column name, and each value is a list of column values.

b. Syntax for Creating DataFrames from Different Sources:

- CSV File: `df = pd.read_csv('file.csv')` – Reads data from a CSV file.
- Excel File: `df = pd.read_excel('file.xlsx')` – Reads data from an Excel file.
- SQL Database: `df = pd.read_sql(query, connection)` – Reads data from a database query.

✓ 3. Essential Operations in Pandas

Now that we understand Series and DataFrames, let's explore some common operations for handling data in a DataFrame.

✓ A. Selecting, Filtering, and Modifying Data

✓ I. Selecting Data:

You can retrieve specific rows, columns, or individual cells.

✓ a. Accessing a Column:

```
# Creating a DataFrame from a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
df['Name'] # Step 1: Access a single column using its name.
```

```
➡ 0      Alice
   1       Bob
   2    Charlie
   Name: Name, dtype: object
```

✓ b. Accessing Multiple Columns:

```
# Creating a DataFrame from a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
df[['Name', 'City']] # Step 2: Access multiple columns by providing a list of column names
```



	Name	City
0	Alice	New York
1	Bob	Los Angeles
2	Charlie	Chicago

✓ c. Accessing Rows Using `iloc`, `loc`, `at` and `iat`:

✓ 1. `loc`: Label-Based Indexing:

- `loc` is used for label-based indexing.
- This means you can use the labels (index or column names) to access rows and columns.
- `loc` is inclusive of the end index, so it will include both the start and end of the specified range.

• Syntax:

```
DataFrame.loc[row_label, column_label]
```

- Example: Using `loc` to Select Rows and Columns by Labels

```
import pandas as pd

# Sample DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}
df = pd.DataFrame(data, index=['a', 'b', 'c', 'd']) # Custom index labels

# Select a single row by label
row_a = df.loc['a']
print("Row 'a':\n", row_a)

# Select multiple rows and specific columns by labels
rows_bc_age_city = df.loc[['b', 'c'], ['Age', 'City']]
print("\nRows 'b' and 'c' with 'Age' and 'City' columns:\n", rows_bc_age_city)

# Select rows based on a condition
age_above_30 = df.loc[df['Age'] > 30, 'Name']
print("\nNames of people older than 30:\n", age_above_30)
```



```
Row 'a':
  Name    Alice
  Age      25
  City  New York
```

Name: a, dtype: object

Rows 'b' and 'c' with 'Age' and 'City' columns:

	Age	City
b	30	Los Angeles
c	35	Chicago

Names of people older than 30:

c	Charlie
d	David

Name: Name, dtype: object

Explanation:

- `loc`: Access rows by index label.
- `df.loc['a']`: Selects all columns in row 'a' (Alice's details).
- `df.loc[['b', 'c'], ['Age', 'City']]`: Selects rows with labels 'b' and 'c', displaying only the 'Age' and 'City' columns.
- `df.loc[df['Age'] > 30, 'Name']`: Uses a condition to select rows where 'Age' is greater than 30, displaying only the 'Name' column.

✓ 2. `iloc`: Position-Based Indexing

- `iloc` is used for integer-location-based indexing.
- This means you specify row and column positions (0-based index) rather than labels.
- `iloc` is also inclusive of the starting index but exclusive of the ending index (like Python slicing).
- Syntax:

```
DataFrame.iloc[row_position, column_position]
```

- Example: Using `iloc` to Access Data by Position

```
import pandas as pd
```

```
# Sample DataFrame
```

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}
```

```
df = pd.DataFrame(data, index=['a', 'b', 'c', 'd']) # Custom index labels
```

```
# Select a single row by position
```

```
row_0 = df.iloc[0]
print("Row 0:\n", row_0)
```

```
# Select multiple rows and specific columns by position
```

```
rows_1_2_columns_1_2 = df.iloc[1:3, 1:3]
```

```
print("\nRows 1 to 2, Columns 1 to 2:\n", rows_1_2_columns_1_2)

# Select a specific value by row and column position
value_row_2_col_1 = df.iloc[2, 1]
print("\nValue at Row 2, Column 1:", value_row_2_col_1)
```

```
⇒ Row 0:
  Name      Alice
  Age        25
  City    New York
Name: a, dtype: object

Rows 1 to 2, Columns 1 to 2:
   Age      City
b   30  Los Angeles
c   35    Chicago

Value at Row 2, Column 1: 35
```

Explanation:

- `df.iloc[0]` : Selects all columns in the first row (row 0).
- `df.iloc[1:3, 1:3]` : Selects rows 1 and 2 and columns 1 and 2, inclusive of the start but exclusive of the end.
- `df.iloc[2, 1]` : Retrieves the value in the third row (row index 2) and second column (column index 1).

Comparison and Best Use Cases for loc, iloc, at, and iat

Method	Access Type	Best Use Case
loc	Label-based	Selecting ranges, specific rows/columns by label
iloc	Position-based	Selecting ranges, specific rows/columns by position

✓ II. Filtering Techniques in Pandas:

Filtering is crucial when working with data as it allows you to select rows that match specific conditions. Here’s a comprehensive look at filtering techniques in Pandas, with examples to illustrate each approach.

✓ 1. Boolean Indexing with Conditions:

Boolean indexing lets you filter rows by applying conditions directly to columns. This technique is useful for straightforward conditional filters.

```
import pandas as pd

# Sample data
```



```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Age': [24, 30, 35, 22, 28],
    'Salary': [50000, 54000, 61000, 45000, 52000]
}
df = pd.DataFrame(data)

# Filter rows where Age is greater than 25
df_age_above_25 = df[df['Age'] > 25]
print("\nRows with Age greater than 25:\n", df_age_above_25)

# Filter rows with multiple conditions: Age > 25 and Salary > 50000
df_multi_condition = df[(df['Age'] > 25) & (df['Salary'] > 50000)]
print("\nRows with Age > 25 and Salary > 50000:\n", df_multi_condition)
```



Rows with Age greater than 25:

	Name	Age	Salary
1	Bob	30	54000
2	Charlie	35	61000
4	Eva	28	52000

Rows with Age > 25 and Salary > 50000:

	Name	Age	Salary
1	Bob	30	54000
2	Charlie	35	61000
4	Eva	28	52000

Explanation:

- `df[df['Age'] > 25]`: Filters rows where the Age column is greater than 25.
- `df[(df['Age'] > 25) & (df['Salary'] > 50000)]`:
 - Filters rows where Age is greater than 25 and Salary is more than 50,000.
 - The & operator is used to apply multiple conditions.

✓ 2. Filtering with query():

The `query()` method offers a more readable way to filter rows with conditions.

```
# Filter rows where Age is greater than 25
df_filtered = df.query('Age > 25')
print("\nRows with Age greater than 25 using query():\n", df_filtered)

# Filter rows where Salary is above 50000
df_high_salary = df.query('Salary > 50000')
print("\nRows with Salary greater than 50000 using query():\n", df_high_salary)
```



Rows with Age greater than 25 using query():

	Name	Age	Salary
--	------	-----	--------

1	Bob	30	54000
2	Charlie	35	61000
4	Eva	28	52000

Rows with Salary greater than 50000 using query():

	Name	Age	Salary
1	Bob	30	54000
2	Charlie	35	61000
4	Eva	28	52000

Explanation:

- `df.query('Age > 25')`:
 - Filters rows where Age is greater than 25.
 - The `query()` method allows a clean and concise syntax.
- `df.query('Salary > 50000')`:
 - Filters rows where Salary is greater than 50,000.
 - This approach is particularly useful for complex conditions.

✓ 3. Filtering Using `isin()` for Multiple Values:

The `isin()` method is ideal for filtering rows based on a set of specific values in a column.

```
# Filter rows where Name is either 'Alice' or 'David'
df_names = df[df['Name'].isin(['Alice', 'David'])]
print("\nRows with Name 'Alice' or 'David':\n", df_names)

# Filter rows where Age is either 24, 30, or 35
df_specific_ages = df[df['Age'].isin([24, 30, 35])]
print("\nRows with specific ages (24, 30, 35):\n", df_specific_ages)
```



Rows with Name 'Alice' or 'David':

	Name	Age	Salary
0	Alice	24	50000
3	David	22	45000

Rows with specific ages (24, 30, 35):

	Name	Age	Salary
0	Alice	24	50000
1	Bob	30	54000
2	Charlie	35	61000

Explanation:

- `df[df['Name'].isin(['Alice', 'David'])]`:
 - Filters rows where the Name column contains either "Alice" or "David".
- `df[df['Age'].isin([24, 30, 35])]`:

- Filters rows where Age matches any value in [24, 30, 35].
- This is useful for filtering specific sets of categorical or numeric values.

✓ III.Modifying Data:

You can add new columns or update values:

```
# Creating a DataFrame from a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
df['Discount'] = df['Age'] * 2 # Adding a new column
```

df



	Name	Age	City	Discount
0	Alice	24	New York	48
1	Bob	27	Los Angeles	54
2	Charlie	22	Chicago	44

✓ B.Handling Missing Data:

- Missing data is common in real-world datasets. Pandas provides functions to identify and handle missing values.

✓ 1.Identifying Missing Data:

- Use `isna()` or `isnull()`:

```
import numpy as np
# Creating a DataFrame from a dictionary
data = {
    'Name': ['Alice', 'Bob', np.nan],
    'Age': [24, 27, 22],
    'City': ['New York', np.nan, 'Chicago']
}
df = pd.DataFrame(data)
df.isna() # Check for missing values
```



	Name	Age	City
0	False	False	False
1	False	False	True
2	True	False	False

Explanation:

`df.isna()` returns a DataFrame of the same size as `df`, with `True` where data is missing and `False` where it's present.

✓ 2. Filling Missing Values (Imputation):

- When dealing with missing data, you can replace NaN values with the mean, median, or mode of a column.

Creating a DataFrame from a dictionary

```
df_with_nans = pd.DataFrame({'Score': [80, 90, None, 95, None]}) # Step 1: Create a Data
mean_imputed = df_with_nans['Score'].fillna(df_with_nans['Score'].mean()) # Step 2: Fill
print("Filled with Mean:\n", mean_imputed)
```

```
median_imputed = df_with_nans['Score'].fillna(df_with_nans['Score'].median()) # Step 3:
print("Filled with Median:\n", median_imputed)
```

```
mode_imputed = df_with_nans['Score'].fillna(df_with_nans['Score'].mode()[0]) # Step 4: F
print("Filled with Mode:\n", mode_imputed)
```



Filled with Mean:

```
0    80.000000
1    90.000000
2    88.333333
3    95.000000
4    88.333333
```

Name: Score, dtype: float64

Filled with Median:

```
0    80.0
1    90.0
2    90.0
3    95.0
4    90.0
```

Name: Score, dtype: float64

Filled with Mode:

```
0    80.0
1    90.0
2    80.0
3    95.0
4    80.0
```

Name: Score, dtype: float64

Explanation:

- `fillna(df_with_nans['Score'].mean())`: Replaces missing values in Score with the column's mean.
- `fillna(df_with_nans['Score'].median())`: Replaces missing values with the median of Score.
- `fillna(df_with_nans['Score'].mode()[0])`: Replaces missing values with the most common value (mode) of Score.

✓ 3. Removing Rows/Columns with Missing Data:

- `dropna()`: Removes rows or columns with missing values.

```
df.dropna(inplace=True) # Drop rows with any missing values
df
```



	Name	Age	City	Discount
0	Alice	24	New York	48
1	Bob	27	Los Angeles	54
2	Charlie	22	Chicago	44

Explanation:

- `dropna()` removes rows that have at least one missing value.
- `inplace=True` makes this change directly in `df`.

✓ C.Data Transformation Techniques:

Pandas makes it easy to sort, rename, and modify columns for better data clarity and usability.

✓ 1.Sorting Data: `sort_values()` and `sort_index()`:

These functions allow you to order data either by columns or by the index.

```
# Sort by Age in descending order
df_sorted_age = df.sort_values(by='Age', ascending=False)
print("\nDataFrame sorted by Age (descending):\n", df_sorted_age)
```

```
# Sort by index
df_sorted_index = df.sort_index()
print("\nDataFrame sorted by index:\n", df_sorted_index)
```



DataFrame sorted by Age (descending):

	Name	Age	City
b	Bob	27	Los Angeles
a	Alice	24	New York
c	Charlie	22	Chicago

DataFrame sorted by index:

	Name	Age	City
a	Alice	24	New York
b	Bob	27	Los Angeles
c	Charlie	22	Chicago

Explanation:

- `df.sort_values(by='Age', ascending=False)`: Sorts DataFrame by 'Age' in descending order. Useful for ranking or ordering data.
- `df.sort_index()`: Orders the DataFrame by index, which can help restore the original order after sorting by other columns.

✓ 2. Renaming Columns: Rename columns for clarity.

```
df.rename(columns={'Discount': 'Discounted Age'}, inplace=True)
df
```



	Name	Age	City	Discounted Age
0	Alice	24	New York	48
1	Bob	27	Los Angeles	54
2	Charlie	22	Chicago	44

Explanation:

- `rename(columns={'Discount': 'Discounted Age'})` renames the Age column to Years.
- Using `inplace=True` makes the change directly in df.

✓ D. Basic Descriptive Statistics:

Pandas provides quick ways to get summary statistics for numeric data in your DataFrame.

✓ 1. `describe()` Method:

- Purpose: Provides summary statistics for numeric columns in a DataFrame.
- Default Behavior:
 - Analyzes only numeric columns by default.

- Can be used on non-numeric columns by passing `include='all'`.

- Example Code:

```
print(df.describe())
```



	Age	Discounted Age
count	3.000000	3.000000
mean	24.333333	48.666667
std	2.516611	5.033223
min	22.000000	44.000000
25%	23.000000	46.000000
50%	24.000000	48.000000
75%	25.500000	51.000000
max	27.000000	54.000000

Explanation:

- Count : Number of non-missing values.
- Mean : The average of the numeric column.
- Standard Deviation (std) : The spread of the data.
- Minimum (min) : The smallest value in the column.
- 25% : The first quartile (25th percentile).
- 50% (median) : The second quartile (50th percentile).
- 75% : The third quartile (75th percentile).
- Maximum (max) : The largest value in the column.

✓ 2. info() Method:

- Purpose: Provides a concise summary of the DataFrame's structure.
- Example Code:

```
df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Name            3 non-null     object
1   Age             3 non-null     int64
2   City            3 non-null     object
3   Discounted Age  3 non-null     int64
dtypes: int64(2), object(2)
memory usage: 228.0+ bytes
```

- Output Details:

- **Class Type** : Indicates the type of the object (e.g., <class 'pandas.core.frame.DataFrame'>).
- **Index Range** : The range of the DataFrame's index (e.g., RangeIndex: 100 entries, 0 to 99).
- **Column Summary**:
 - **Total Columns** : Number of columns in the DataFrame.
 - **Column Names and Data Types** : List of columns, the number of non-null entries, and their data types (e.g., int64, float64, object, bool, datetime64).
 - **Non-Null Count** : The number of non-missing entries in each column.
 - **Data Types** : Shows the count of different data types used in the DataFrame.
 - **Memory Usage** : Displays the memory footprint of the DataFrame.

✓ E. Working with Data Types and Memory Optimization:

Pandas allows you to modify data types for memory optimization and to save data for future use.

✓ 1.Changing Data Types:

```
df['Age'].astype(float) # Convert 'Age' column to float.
```

```
⇒ 0    24.0  
   1    27.0  
   2    22.0  
   Name: Age, dtype: float64
```

Explanation:

- `astype(float)` converts the Years column to a floating-point format.

```
# Sample data for inspection  
data = {'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Chicago']}  
df = pd.DataFrame(data)
```

```
# Get unique values in 'City' column  
print("Unique cities:", df['City'].unique())
```

```
# Count occurrences of each city  
print("\nCity counts:\n", df['City'].value_counts())
```

```
⇒ Unique cities: ['New York' 'Los Angeles' 'Chicago']  
  
City counts:  
City  
New York      2
```



```
Chicago      2
Los Angeles  1
Name: count, dtype: int64
```

Explanation and Application:

- `df['City'].unique()` : Identifies all unique values in the 'City' column. Useful for categorical analysis.
- `df['City'].value_counts()` : Counts occurrences of each unique value in 'City', making it easy to see how frequently each category appears.

✓ F. Data Inspection: `unique()` and `value_counts()`:

These functions are great for examining the values within a specific column.

```
import pandas as pd

# Sample data
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank'],
        'Age': [25, 30, 35, 40, 45, 50]}
df = pd.DataFrame(data)

# Display first 3 rows
print("First 3 rows:\n", df.head(3))

# Display last 3 rows
print("\nLast 3 rows:\n", df.tail(3))

# Display a random sample of 2 rows
print("\nRandom sample of 2 rows:\n", df.sample(2))

# Get the shape of the DataFrame
print("\nShape of DataFrame:", df.shape)

# Get the total number of elements
print("Total elements in DataFrame:", df.size)
```

➡ First 3 rows:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35

Last 3 rows:

	Name	Age
3	David	40
4	Eva	45
5	Frank	50

Random sample of 2 rows:

	Name	Age
--	------	-----

```
1 Bob 30
4 Eva 45
```

```
Shape of DataFrame: (6, 2)
Total elements in DataFrame: 12
```

Explanation and Application:

- `df.head(3)` : Displays the first 3 rows of the DataFrame, giving an idea of the structure of the data.
- `df.tail(3)` : Displays the last 3 rows, useful for checking the end of the data.
- `df.sample(2)` : Randomly samples 2 rows, helping get a quick sense of data diversity.
- `df.shape` : Returns a tuple of the (number of rows, number of columns), providing a basic understanding of the DataFrame's size.
- `df.size` : Returns the total number of elements, or the product of rows and columns.

✓ **G. Viewing Data: `head()`, `tail()`, `sample()`, `shape`, and `size`:**

These functions provide an initial overview of the data structure.

```
import pandas as pd

# Sample data
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank'],
        'Age': [25, 30, 35, 40, 45, 50]}
df = pd.DataFrame(data)

# Display first 3 rows
print("First 3 rows:\n", df.head(3))

# Display last 3 rows
print("\nLast 3 rows:\n", df.tail(3))

# Display a random sample of 2 rows
print("\nRandom sample of 2 rows:\n", df.sample(2))

# Get the shape of the DataFrame
print("\nShape of DataFrame:", df.shape)

# Get the total number of elements
print("Total elements in DataFrame:", df.size)
```

```
➡ First 3 rows:
   Name  Age
0  Alice   25
1   Bob   30
2 Charlie   35
```

```
Last 3 rows:
   Name  Age
4   Eva   45
5  Frank   50
```

```
3 David 40
4 Eva 45
5 Frank 50
```

Random sample of 2 rows:

```
      Name Age
3 David  40
1 Bob    30
```

Shape of DataFrame: (6, 2)

Total elements in DataFrame: 12

Explanation and Application:

- `df.head(3)` : Displays the first 3 rows of the DataFrame, giving an idea of the structure of the data.
- `df.tail(3)` : Displays the last 3 rows, useful for checking the end of the data.
- `df.sample(2)` : Randomly samples 2 rows, helping get a quick sense of data diversity.
- `df.shape` : Returns a tuple of the (number of rows, number of columns), providing a basic understanding of the DataFrame's size.
- `df.size` : Returns the total number of elements, or the product of rows and columns.

✓ H. Index and Column Management: `set_index()`, `reset_index()`, and `rename()`:

These functions help in controlling the indexing and renaming of DataFrame columns.

```
# Set 'Name' column as index
df_indexed = df.set_index('Name')
print("\nDataFrame with 'Name' as index:\n", df_indexed)

# Reset index back to default integers
df_reset = df_indexed.reset_index()
print("\nDataFrame after resetting index:\n", df_reset)

# Rename columns for clarity
df_renamed = df.rename(columns={'Salary': 'Annual Salary'})
print("\nDataFrame with renamed columns:\n", df_renamed)
```



DataFrame with 'Name' as index:

```
      Age
Name
Alice  25
Bob    30
Charlie 35
David  40
Eva    45
Frank  50
```

DataFrame after resetting index:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35
3	David	40
4	Eva	45
5	Frank	50

DataFrame with renamed columns:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35
3	David	40
4	Eva	45
5	Frank	50

Explanation and Application:

- `df.set_index('Name'):`
 - Sets the specified column ('Name') as the index of the DataFrame.
 - Setting an index is useful for making specific columns the primary reference point, often used in time-series data or unique identifiers.
- `df.reset_index():`
 - Resets the index back to the default integer index, and the previous index becomes a column in the DataFrame.
 - This can be helpful if you need to restore the original indexing after setting a column as an index.
- `df.rename(columns={'Salary': 'Annual Salary'}):`
 - Renames columns in the DataFrame, making them clearer and more descriptive.
 - This is useful for aligning column names with their content or adjusting names for presentation.

THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)